```
import java.util.Date;
import java.util.UUID;

// ========================================
// 1. Data Transfer Objects (DTO)
// MUHAMMAD SAMUDERA BAGJA
// ========================================
class PaymentRequest {
    public String orderId;
    public double amount;
    public String currency;
    public String paymentMethod;
    public String customerEmail;

    public PaymentRequest(String orderId, double amount, String currency, String p...
        this.orderId = orderId;
        this.amount = amount;
        this.currency = currency;
        this.paymentMethod = paymentMethod;
        this.customerEmail = customerEmail;
    }
}

class PaymentResponse {
    public String transactionId;
    public String status;
    public Date timestamp;

    public PaymentResponse(String transactionId, String status) {
        this.transactionId = transactionId;
        this.status = status;
        this.timestamp = new Date();
    }
}

// ========================================
// 2. Strategy Interface
// ========================================
interface PaymentStrategy {
    PaymentResponse pay(PaymentRequest req);
}

// ========================================
// 3. Concrete Strategies
// ========================================
class MidtransVAStrategy implements PaymentStrategy {
    private String serverKey;

    public MidtransVAStrategy(String serverKey) {
        this.serverKey = serverKey;
    }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
```

```
        System.out.println("[Midtrans] Processing payment via Virtual Account...");
        String vaNumber = generateVANumber();
        System.out.println("[Midtrans] VA Number for " + req.customerEmail + " : "...

        return new PaymentResponse(UUID.randomUUID().toString(), "PENDING_PAYMENT");
    }

    private String generateVANumber() {
        // Mock VA generation logic
        return "8077" + (int) (Math.random() * 10000000);
    }
}

class StripeCreditCardStrategy implements PaymentStrategy {
    private String secretKey;

    public StripeCreditCardStrategy(String secretKey) {
        this.secretKey = secretKey;
    }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
        System.out.println("[Stripe] Processing Credit Card payment of " + req.amo...

        if (validate3DSecure()) {
            System.out.println("[Stripe] 3D Secure Valid! Payment Successful.");
            return new PaymentResponse(UUID.randomUUID().toString(), "SUCCESS");
        } else {
            System.out.println("[Stripe] 3D Secure Failed.");
            return new PaymentResponse(UUID.randomUUID().toString(), "FAILED");
        }
    }

    private boolean validate3DSecure() {
        // Mock OTP/3DSecure validation
        return true;
    }
}

class XenditEWalletStrategy implements PaymentStrategy {
    private String apiKey;

    public XenditEWalletStrategy(String apiKey) {
        this.apiKey = apiKey;
    }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
        System.out.println("[Xendit] Processing E-Wallet for Order: " + req.orderId);
        sendPushNotification();
        return new PaymentResponse(UUID.randomUUID().toString(), "WAITING_USER_CON...
    }

    private void sendPushNotification() {
```

```
        System.out.println("[Xendit] Sending Push Notification to User's Phone...");
    }
}

class CryptoWalletStrategy implements PaymentStrategy {
    private String networkRpcUrl;

    public CryptoWalletStrategy(String networkRpcUrl) {
        this.networkRpcUrl = networkRpcUrl;
    }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
        System.out.println("[Crypto] Processing On-Chain payment...");

        if (verifyOnChainTransaction()) {
            System.out.println("[Crypto] Transaction confirmed on node network: " ...
            return new PaymentResponse(UUID.randomUUID().toString(), "SUCCESS");
        }
        return new PaymentResponse(UUID.randomUUID().toString(), "FAILED");
    }

    private boolean verifyOnChainTransaction() {
        // Mock blockchain verification
        return true;
    }
}

// ========================================
// 4. Context Class
// ========================================
class PaymentProcessor {
    private PaymentStrategy strategy;

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public PaymentResponse process(PaymentRequest req) {
        if (this.strategy == null) {
            throw new IllegalStateException("Payment Strategy has not been set!");
        }
        return this.strategy.pay(req);
    }
}

// ========================================
// 5. Client / Controller Layer
// ========================================
class PaymentController {
    public PaymentResponse handleCheckout(PaymentRequest req) {
        PaymentProcessor processor = new PaymentProcessor();

        // Select payment strategy based on request input
```

```
        switch (req.paymentMethod.toUpperCase()) {
            case "MIDTRANS_VA":
                processor.setStrategy(new MidtransVAStrategy("server-key-md-123"));
                break;
            case "STRIPE_CC":
                processor.setStrategy(new StripeCreditCardStrategy("sk_test_4eC39H...
                break;
            case "XENDIT_EWALLET":
                processor.setStrategy(new XenditEWalletStrategy("xnd_live_abc123"));
                break;
            case "CRYPTO_WEB3":
                processor.setStrategy(new CryptoWalletStrategy("https://eth-mainne...
                break;
            default:
                throw new IllegalArgumentException("Payment method not supported: ...
        }

        // Execute payment via Context
        return processor.process(req);
    }
}

// ==========================================
// 6. Main Executable (Demo)
// ==========================================
public class PaymentGatewayDemo {
    public static void main(String[] args) {
        System.out.println("=== E-Commerce Checkout System ===");
        PaymentController controller = new PaymentController();

        // Scenario 1: Pay using Midtrans Virtual Account
        PaymentRequest req1 = new PaymentRequest("ORD-001", 150000, "IDR", "MIDTRA...
        PaymentResponse res1 = controller.handleCheckout(req1);
        System.out.println("Transaction 1 Status: " + res1.status + "\n");

        // Scenario 2: Pay using Stripe Credit Card
        PaymentRequest req2 = new PaymentRequest("ORD-002", 45.50, "USD", "STRIPE_...
        PaymentResponse res2 = controller.handleCheckout(req2);
        System.out.println("Transaction 2 Status: " + res2.status + "\n");

        // Scenario 3: Pay using Crypto
        PaymentRequest req3 = new PaymentRequest("ORD-003", 0.05, "ETH", "CRYPTO_W...
        PaymentResponse res3 = controller.handleCheckout(req3);
        System.out.println("Transaction 3 Status: " + res3.status + "\n");
    }
}
```